Lab 11: Automated Unit Testing and Coverage Analysis

Library Management System

1. Objective

In this laboratory, you act as a Software Quality Assurance (QA) and Test Engineer. You are provided with a complete, backend-only Library Management System that simulates the core workflows of institutional and academic libraries: catalog management, multi-copy inventory, tier-based lending rules, hold queues, overdue fine calculation, and payment settlement. Your task is to:

- Analyse and study the modular architecture and business logic of each module.
- Develop automated test suites using pytest without modifying any production source code.
- Achieve at least 90% line coverage and at least 90% branch coverage using pytest-cov / coverage.

Source of Code. The library_management code used in this lab is the code attached in the Google Classroom assignment. You must use it as it is, without any modification.

2. Environment Setup

From the library-management root directory, install the test dependencies and verify the tools:

```
pip install -r requirements.txt

pytest --version
coverage --version
```

3. Codebase Architecture and Study Guide

All source code is located under src/library_management/. Study the modules in the following sequence before writing any tests.

Step	Module	File	Core concepts to inspect
1	Exceptions	exceptions.py	Custom error hierarchy derived from LibraryManagementError. Every business failure (e.g. LoanLimitExceededError, MemberSuspendedError, OutstandingFinesBlockError) raises a specific exception.
2	Models and Enums	models.py	Dataclasses (Member, Book, BookCopy, Loan, Reservation, FineRecord, Transaction) and enums (MembershipType, MemberStatus, BookFormat, BookStatus, LoanStatus, ReservationStatus, PaymentMethod).
3	Utilities	utils.py	Clock (injectable for time travel), IdGenerator (deterministic sequences), validation patterns for email and phone, and checksum verification for ISBN-10 and ISBN-13.
4	Member Management	member.py	Member registration across tiers (STUDENT, FACULTY, PUBLIC, RESEARCHER), account balances, validity periods, suspensions and reactivations.
5	Catalog Management	catalog.py	Book cataloguing, physical copy barcode generation, rack location assignment, availability statuses, multi-criteria search (title, author, category).
6	Fine Engine	fine.py	Overdue fine calculation with 2-day grace period, daily tier rates, maximum fine cap (₹500), lost book replacement (price + ₹100 fee), and fine waivers.

Step	Module	File	Core concepts to inspect
7	Lending Management	lending.py	Checkout rules (loan limits per tier, loans blocked when fines exceed ₹100, reference-only restrictions), renewals, returns and lost book reporting.
8	Reservation and Holds	reservation.py	Waitlist queue management, FIFO hold allocation on return, 3-day pickup deadlines, cancellation and hold expiration cleanup.
9	Payment Management	payment.py	PaymentGateway interface, MockPaymentGateway with failure triggers, prepaid account debits, cash desk collection, card/UPI gateway charges and ledger tracking.
10	Notifications	notification.py	Mock SMS and Email delivery channels, checkout confirmations, hold pickup alerts, overdue notices and audit logging.
11	Analytics	analytics.py	Inventory utilisation, circulation statistics, top borrowed books aggregation and fine revenue totals.
12	System Facade	system.py	LibraryManagementSystem orchestrating member onboarding, cataloguing, checkout, return-with-hold-check and fine settlement for end-to-end integration tests.

4. Writing Unit Tests

4.1 Required Test Layout

Create a tests/ directory at the project root with the following structure:

```
library-management/
├── src/
│   └── library_management/
├── tests/
│   ├── conftest.py
│   ├── test_member.py
│   ├── test_catalog.py
│   ├── test_fine.py
│   ├── test_lending.py
│   ├── test_reservation.py
│   ├── test_payment.py
│   ├── test_notification.py
│   ├── test_analytics.py
│   └── test_system.py
└── requirements.txt
```

4.2 Testing Strategies

- Happy paths and error branches. Test normal borrowing and returns as well as edge-case blocks (borrowing with unpaid fines above ₹100, exceeding loan limits, borrowing reference-only books). Test validation errors such as malformed ISBN checksums, negative payments and duplicate registrations.
- Deterministic time injection. Use Clock and clock.advance_days(...) to simulate due-date expiry, testing overdue calculations across member tiers, grace periods and reservation pickup window expiries.
- Mock interfaces. Use MockPaymentGateway with set_failure_trigger(...) to test payment failure branches. Use MockSmsService and MockEmailService with should_succeed=False to exercise notification failure handling.
- Boundary values. Test the fine cap (₹500), loan limit boundaries (4 books for Student, 10 for Faculty), maximum renewals (2 for Student, 5 for Faculty), and the ISBN-10 check digit 'X'.
- Fixtures. Place shared fixtures (clock, members, books, mock gateway) in conftest.py.

5. Running Tests and Generating Coverage Reports

Run the following commands from the library-management directory.

5.1 Standard test run

```
pytest
```

5.2 Line and branch coverage in the terminal

```
pytest --cov=src/library_management --cov-branch --cov-report=term-missing
```

5.3 Interactive HTML coverage report

```
pytest --cov=src/library_management --cov-branch --cov-report=html
```

Open htmlcov/index.html in a browser to inspect lines and branches that were missed. Missed lines are highlighted in red and partially covered branches in yellow.

6. Submission Requirements

Submit a single PDF report. The report must contain:

- A test case table listing, for each test module, the test ID, the module and behaviour under test, the input or setup, the expected result and the actual result (pass or fail).
- Screenshots of the terminal output from the coverage command in Section 5.2, showing the line and branch coverage percentages (must be > 90%) and the missed-lines column.
- A screenshot of the terminal output from the standard pytest run in Section 5.1, showing the pass count.

Name the report using your student ID, for example 202301234_Lab11_Report.pdf.